Coding — Data Structures & Algorithms Notes

PlacementPrep Study Notes

1. Time Complexity Basics

Big-O describes how runtime grows with input size 'n'.

- O(1) constant, O(log n) logarithmic, O(n) linear, O(n log n), O(n^2) quadratic.
- Binary search on a sorted array runs in O(log n).
- Nested loops over the same input typically give O(n^2).

2. Arrays & Strings

Most interview problems start here — know these patterns.

- Two-pointer technique: useful for sorted array pair-sum problems.
- Sliding window: useful for subarray/substring problems with a size or sum constraint.
- Prefix sums: precompute cumulative sums for fast range-sum queries.

3. Stacks & Queues

Stack = LIFO (Last In, First Out). Queue = FIFO (First In, First Out).

- Stacks are used for balanced parentheses checks and undo operations.
- Queues are used in breadth-first search (BFS) and task scheduling.

4. Trees & Graphs

A Binary Search Tree keeps left < root < right at every node.

- Inorder traversal of a BST gives elements in sorted order.
- BFS explores level by level using a queue; DFS explores depth-first using a stack or recursion.

5. Dynamic Programming

Break a problem into overlapping subproblems and store results.

- Memoization: top-down, caches results of recursive calls.
- Tabulation: bottom-up, fills a table iteratively.
- Classic examples: Fibonacci, Knapsack, Longest Common Subsequence.

Practice Tip

- Always state the time and space complexity out loud when solving a problem in an interview.
- Practice explaining your approach before writing code — interviewers value clear thinking.